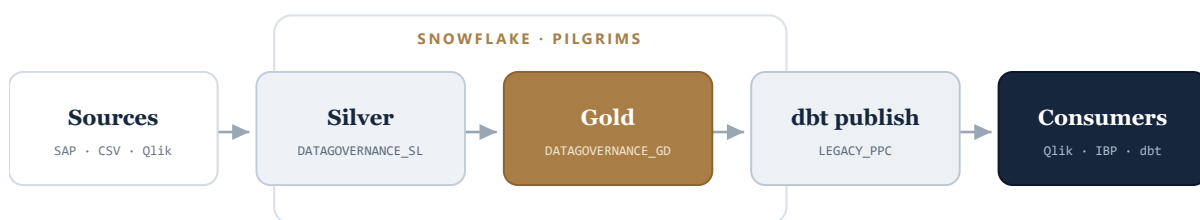


Master Data Documentation

The Snowflake & dbt master-data platform — architecture, tables, and the stored-procedure API.

PLATFORM LINEAGE



What's inside

01	Architecture	3
02	Environments: PROD and DEV	4
03	How data gets in	4
04	The dbt layer	5
05	The tables	7
06	Stored procedures	10
07	Conventions & validation	14
08	Operational notes & gotchas	17
09	Glossary	18

Architecture

15

GOLD TABLES

31

STORED PROCEDURES

19

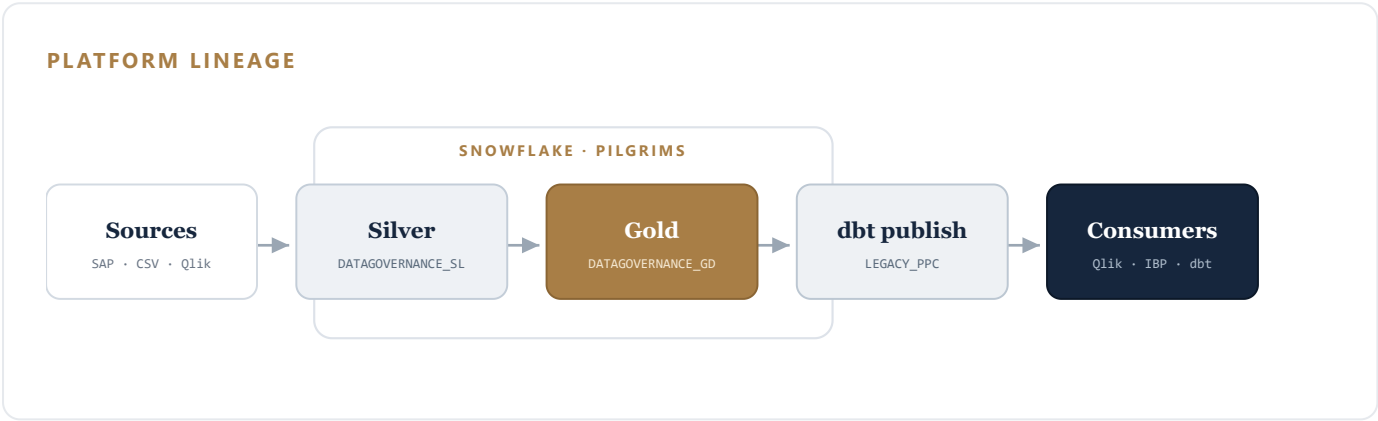
LOV VOCABULARIES

171K

LARGEST MAPPING · ROWS

06:00

DAILY REFRESH · MST



The data flows left to right: **Sources** → **Silver** → **Gold** → **dbt publish** → **Consumers**.

STAGE	LOCATION	ROLE
Sources	SAP CO-PA / customer / material; curated CSV mapping files; Qlik write-back & Data-Governance app	Raw inputs and interactive edits.
Silver	PILGRIMS.DATAGOVERNANCE_SL — MAPPING_*, LOV_*	Editable source of truth. Written by the daily 06:00 MST batch ETL (bulk) and by the SP_INSERT_* procedures (interactive).
Gold	PILGRIMS.DATAGOVERNANCE_GD — CONSOLIDATED_ACCOUNT_LIST, D_*, LOV* (15 tables)	Curated, deduplicated, code-resolved dimensions. Rebuilt / merged from silver.
dbt publish	LEGACY_PPC.PPC_MASTER_DATA	Read-only governed copy of gold (SELECT *).
Consumers	Qlik Sense, Dataiku P&L, IBP, dbt analytics	Read the published layer.

Three layers, one principle: **silver is editable, gold is curated, dbt republishes gold read-only**. Each layer has a **PROD** and a **DEV** copy (see §2).

SECTION 02

Environments: PROD and DEV

Everything lives in the `PILGRIMS` database, split into production and development schemas. A parallel set of `*_DEV` schemas mirrors production for testing.

LAYER	PRODUCTION SCHEMA	DEVELOPMENT SCHEMA
Silver (editable source)	<code>PILGRIMS.DATAGOVERNANCE_SL</code>	<code>PILGRIMS.DATAGOVERNANCE_SL_DEV</code>
Gold (curated)	<code>PILGRIMS.DATAGOVERNANCE_GD</code>	<code>PILGRIMS.DATAGOVERNANCE_GD_DEV</code>
Published copy (dbt)	<code>LEGACY_PPC.PPC_MASTER_DATA</code>	<code>DBT.DBT_<user>_PPC_MASTER_DATA (dev), LEGACY_PPC_QA_<id>.PPC_MASTER_DATA (QA)</code>

- The **stored procedures are deployed in both** `DATAGOVERNANCE_SL` (prod) and `DATAGOVERNANCE_SL_DEV` (dev). You choose the environment by **which schema you CALL** — the prod copy writes the prod silver+gold schemas; the dev copy writes the `_DEV` schemas. The schema suffix is hardcoded in each procedure body, so the same procedure cannot be redirected by an argument.
- The dbt models target `LEGACY_PPC` in prod and the shared `DBT` database in dev/CI (see §4).

SECTION 03

How data gets in — two governed write paths

Master Data has **two** write paths into the production governed tables. They are complementary.

3.1 Daily ETL — the bulk refresh (06:00 MST)

The Master Data ETL is a single Snowflake stored procedure, `PILGRIMS.DATAGOVERNANCE_SL.SP_MASTER_DATA_ETL()` (with a dev twin in `DATAGOVERNANCE_SL_DEV`), scheduled to run **daily at 06:00 MST**. It is **orchestrated by dbt**: a macro (`run_master_data_etl`) **CALL**s the procedure on a scheduled dbt job — deliberately *not* a native Snowflake Task, which would have required extra account-level grants. The procedure wraps the entire ETL **in file order** with per-step logging (`MD_ETL_RUN_LOG` / `MD_ETL_STEP_LOG`), and is generated from the source SQL in `Master Data/PROD/ETL.txt` by a deterministic builder.

What it does, at a high level: it ingests SAP-derived sales/customer data and curated CSV mapping inputs (ship-to-to-territory, territory-to-RSM, redistributor, etc.), normalizes the records (descriptions upper-cased and trimmed; surrogate codes auto-generated for any new value, e.g. `CE-0001`, `RSM001`, `T00001`), and then **inserts only the new / not-yet-mapped records** into the silver `DATAGOVERNANCE_SL` tables — existing rows keep their previously assigned codes. After silver is updated it **rebuilds the gold layer** (`DATAGOVERNANCE_GD`) from silver, and finishes by fully rebuilding `CONSOLIDATED_ACCOUNT_LIST`. The practical takeaway for consumers: **new master-data rows appear once per day**, the morning after they exist in the source.

The job is organized into four sequential zones:

#	ZONE	OUTPUT SCHEMA	SCOPE
1	Lists of Values (silver)	DATAGOVERNANCE_SL	19 LOV_* builds: uppercase + assign codes to new values
2	Lists of Values (gold)	DATAGOVERNANCE_GD	4 LOV* tables rebuilt from silver (DELETE + MERGE)
3	Mappings (silver)	DATAGOVERNANCE_SL	10 MAPPING_* builds from CSV + SAP inputs, with enrichment UPDATES
4	Mappings (gold)	DATAGOVERNANCE_GD	10 D_* dimensions + the final CONSOLIDATED_ACCOUNT_LIST rebuild

3.2 Application write-back — the `SP_INSERT_*` procedures

For **interactive, single-row edits**, the Qlik write-back sheets and the Data-Governance application call the `SP_INSERT_*` stored procedures (§6). These exist in both prod and dev and are the **only** supported way to write a single mapping row — never with direct `INSERT / UPDATE / MERGE`. They normalize the input, resolve codes from the LOVs, land the row in silver, and propagate it to gold in one call.

Both paths can write production gold. The daily ETL is the bulk refresh; the SPs handle interactive edits. They are designed to coexist (the gold rebuild logic is idempotent and reconciles from silver).

SECTION 04

The dbt layer (`legacy_ppc / ppc_master_data`)

Core principle: the dbt model is just a governed copy of the gold table into `LEGACY_PPC`. The substantive logic (dedup, MERGE, EEID backfill, reconciliation) lives upstream in the ETL and the `SP_INSERT_*` procedures. dbt does **not** reimplement it — it reads the already-governed `DATAGOVERNANCE_GD` result and republishes it under the `LEGACY_PPC` lineage so legacy Qlik/SQL that hardcodes `LEGACY_PPC.PPC_MASTER_DATA.*` keeps working against a stable, tested, documented entry point.

Folder structure

STRUCTURE

```
models/legacy_ppc/ppc_master_data/
├─ _sources_pilgrims_sl.yml          # source() defs for DATAGOVERNANCE_SL LOVs (enrichment only)
├─ List_of_Values_GD/
│   ├── lovcor_lov_corporate_entity.sql
│   ├── lovdiss_lov_distributor.sql
│   ├── lovepe_lov_operator_account.sql
│   └─ lovrac_redistributor_accounts.sql
└─ Mappings_GD/
    ├── consolidated_account_list.sql
    ├── d_custcl_customer_classification.sql
    ├── d_mapopa_mapping_operator_accounts.sql
    ├── d_mapred_mapping_redistributor.sql
    ├── d_mapren_mapping_redistributor_names.sql
    ├── d_operep_operator_report.sql
    ├── d_prdcat_product_category.sql
    ├── d_remdet_redistributor_mapping_details.sql # enrichment (not a passthrough)
    └─ d_shpexc_shipto_exceptions.sql             # DEV-only source today (JBS-800)
```

Almost every model is a thin `SELECT *` passthrough of one gold table, materialized as a `table` in schema `PPC_MASTER_DATA`. The canonical shape:

SQL

```
-- Model: d_operep_operator_report
-- Pass-through to the deduplicated Operator Report maintained in PILGRIMS.DATAGOVERNANCE_GD.
WITH source AS (
    SELECT * FROM pilgrims.datagovernance_gd.d_operep_operator_report
),
final AS (
    SELECT * FROM source
)
SELECT * FROM final
```

Two deviations from the pure-passthrough rule:

- `d_remdet_redistributor_mapping_details` — the only enrichment model: builds on `ref('d_mapred...')`, expands each redistributor row into Field-Sales / Corporate-Accounts sub-channel rows, and backfills channel/sub-channel/buying-group codes from SL LOVs.
- `d_shpexc_shipto_exceptions` — the only model still reading from **DEV** gold (`DATAGOVERNANCE_GD_DEV`), pending the JBS-800 prod-GD promotion.

Database & schema routing

`legacy_ppc` is a legacy "lift-and-shift" domain: the folder name is the Snowflake **database**, the direct subfolder name (`ppc_master_data`) is the **schema** (no layer prefix, resolved automatically).

ENVIRONMENT	DATABASE	SCHEMA
Dev (local / Cloud IDE)	DBT (shared)	DBT_<user>_PPC_MASTER_DATA
CI (PR build)	DBT (shared)	CI_PR_<id>_PPC_MASTER_DATA
QA	LEGACY_PPC_QA_<build_id> (clone)	PPC_MASTER_DATA
Prod	LEGACY_PPC	PPC_MASTER_DATA

The fully-qualified target is `<database>.<schema>.<table>` — e.g. in production, `LEGACY_PPC.PPC_MASTER_DATA.D_`
`OPERE_P_OPERATOR_REPORT` .

Parity (verified May 2026): all 15 tables in `DATAGOVERNANCE_GD` match `LEGACY_PPC.PPC_MASTER_DATA` byte-for-byte (column structure + row count + `HASH_AGG(*)`). The dbt publish is the maintained successor to that legacy mirror.

SECTION 05

The tables

5.1 Gold layer — `DATAGOVERNANCE_GD` (15 tables; what consumers read)

Gold tables and columns are **prefixed** (`D_CUSTCL_*` / `CUSTCL_*` , etc.) so consumers get stable, self-describing names. Row counts are catalog statistics (approximate).

Dimension / mapping tables (11)

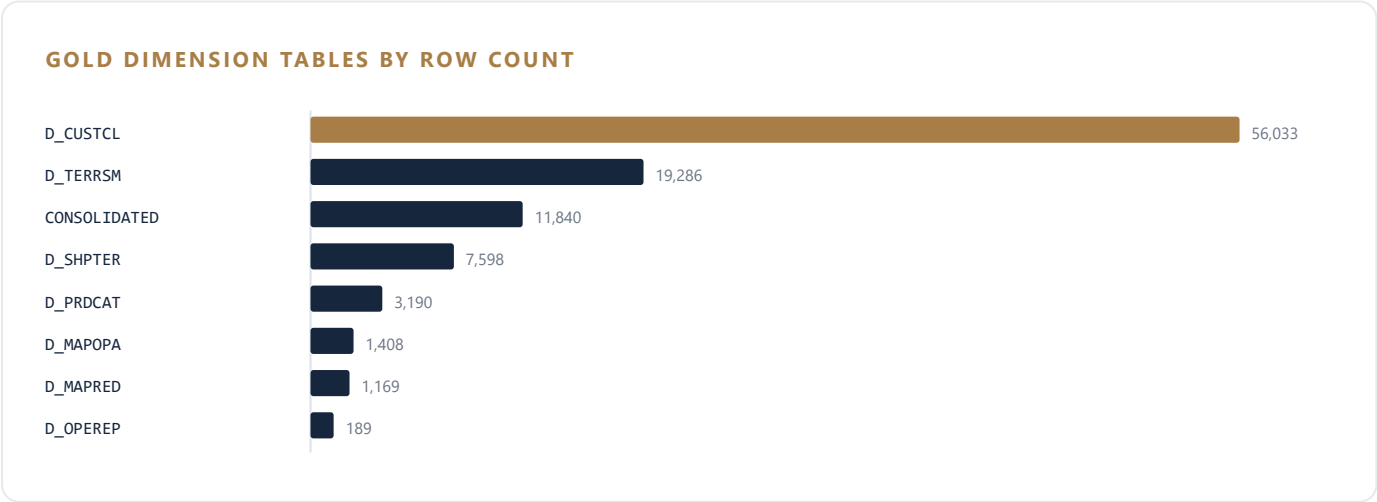


TABLE	PURPOSE	ROWS
CONSOLIDATED_ACCOUNT_LIST	Unified master list of accounts (corporate entity, distributor, redistributor, operator) with address + parent-account hierarchy.	11,840
D_CUSTCL_CUSTOMER_CLASSIFICATION	Core customer classification: ship-to/payer to channel, sub-channel, territory, distributor, redistributor, corporate entity.	56,033
D_MAPOPA_MAPPING_OPERATOR_ACCOUNTS	Operator ship-to to corporate entity, distributor, redistributor account.	1,408
D_MAPRED_MAPPING_REDISTRIBUTOR	Redistributor ship-to mapping (distributor type, buying group, responsibilities, corporate entity).	1,169
D_MAPREN_MAPPING_REDISTRIBUTOR_NAMES	Payer to redistributor account name + corporate entity.	7
D_MGRARE_MANAGER_AREA	Manager to area lookup.	14
D_OPEREP_OPERATOR_REPORT	Operator Report: territory matrix to sales person, RSM/manager (EEIDs), channel/sub-channel, corporate entity, active flag.	189
D_PRDCAT_PRODUCT_CATEGORY	Product category: material to FSE/budget categories + product-family attributes.	3,190
D_RSMGR_RSM_MANAGER	Regional Sales Manager to manager mapping.	47
D_SHPTER_SHIPTO_TERRITORY	Ship-to ZIP + sub-channel to sales territory.	7,598
D_TERRSM_TERRITORY_RSM	Territory + sub-channel to Regional Sales Manager.	19,286

LOV tables (4) — all share the shape `<prefix>_CODE` , `<prefix>_DESCRIPTION` , `<prefix>_ACCOUNT_TYPE` :

TABLE	PURPOSE	ROWS
LOVCOR_LOV_CORPORATE_ENTITY	Valid corporate-entity codes/descriptions.	190
LOVDIS_LOV_DISTRIBUTOR	Valid distributor codes/descriptions.	4,420
LOVOPE_LOV_OPERATOR_ACCOUNT	Valid operator-account codes/descriptions.	7,207
LOVRAC_REDISTRIBUTOR_ACCOUNTS	Valid redistributor-account codes/descriptions.	23

Columns of the main gold dimensions (all `TEXT` unless noted):

- **CONSOLIDATED_ACCOUNT_LIST** (10): `ACCOUNT_CATEGORY` , `ACCOUNT_TYPE` , `ACCOUNT_CODE` , `ACCOUNT_NAME` , `ADDRESS` , `CITY` , `STATE` , `ZIP_CODE` , `DISTRIBUTOR_TYPE` , `PARENT_ACCOUNT_ID` .
- **D_CUSTCL_CUSTOMER_CLASSIFICATION** (23): `CUSTCL_PRODUCT` , `CUSTCL_PRODUCT_DESCRIPTION` , `CUSTCL_BRAND_DESCRIPTION` , `CUSTCL_BRAND` , `CUSTCL_SHIP_TO_PARTY` , `CUSTCL_SHIP_TO_PARTY_DESCRIPTION` , `CUSTCL_SHIP_TO_ZIP_CODE` , `CUSTCL_REGION` , `CUSTCL_PAYER` , `CUSTCL_PAYER_DESCRIPTION` , `CUSTCL_PROFIT_CENTER` , `CUSTCL_CHANNEL_CODE` , `CUSTCL_CHANNEL` , `CUSTCL_SUBCHANNEL_CODE` , `CUSTCL_SUBCHANNEL` , `CUSTCL_REDISTRIBUTOR_ACCOUNT_CODE` , `CUSTCL`

REDISTRIBUTOR_ACCOUNT_DESCRIPTION , CUSTCL_CORPORATE_ENTITY , CUSTCL_CORPORATE_ENTITY_DESCRIPTION , CUSTCL_DISTRIBUTOR , CUSTCL_DISTRIBUTOR_DESCRIPTION , CUSTCL_TERRITORY_CODE , CUSTCL_TERRITORY_DESCRIPTION .

- **D_OPEREP_OPERATOR_REPORT** (12): OPEREP_TERRITORY_MATRIX_NUM (*NUMBER*), RSM_EEID , OPEREP_SALES_PERSON , OPEREP_OPERATOR_STATE , OPEREP_TERRITORY_NAME , OPEREP_CORPORATE_ENTITY , OPEREP_CORPORATE_ENTITY_DESCRIPTION , OPEREP_CHANNEL , OPEREP_SUBCHANNEL , MANAGER_EEID , OPEREP_MANAGER , OPEREP_ACTIVE_FLAG .
- **D_PRDCAT_PRODUCT_CATEGORY** (18): PRDCAT_MATERIAL_CODE , PRDCAT_MATERIAL_DESCRIPTION , PRDCAT_FSE_PRODUCT_CATEGORY_CODE , PRDCAT_FSE_PRODUCT_CATEGORY , PRDCAT_BUDGET_CATEGORY_CODE , PRDCAT_BUDGET_CATEGORY , plus the six PF_* pairs (PROCESS_TYPE , RAW_MATERIAL_CATEGORY , PRODUCT_TYPE , BRAND_TYPE , BREADING_TYPE , MEAT_TYPE), each as a _CODE + label.
- **D_MAPRED_MAPPING_REDISTRIBUTOR** (10): MAPRED_SHIP_TO_NUMBER , MAPRED_SHIP_TO_NAME , MAPRED_DISTRIBUTOR_TYPE , MAPRED_BUYING_GROUP , MAPRED_CORPORATE_ACCOUNT_RESPONSIBLE , MAPRED_FIELD_SALES_RESPONSIBLE , MAPRED_CORPORATE_ENTITY_CODE , MAPRED_CORPORATE_ENTITY_DESCRIPTION , MAPRED_DISTRIBUTOR_CODE , MAPRED_DISTRIBUTOR_DESCRIPTION .
- **D_MAPOPA_MAPPING_OPERATOR_ACCOUNTS** (8): MAPOPA_SHIP_TO , MAPOPA_SHIP_TO_DESCRIPTION , MAPOPA_CORPORATE_ENTITY , MAPOPA_CORPORATE_ENTITY_DESCRIPTION , MAPOPA_DISTRIBUTOR , MAPOPA_DISTRIBUTOR_DESCRIPTION , MAPOPA_REDISTRIBUTOR_ACCOUNT , MAPOPA_REDISTRIBUTOR_ACCOUNT_DESCRIPTION .
- Smaller dimensions: D_SHPTER_* (4: zip, sub-channel, territory code, territory), D_TERRSM_* (5: territory code/name, sub-channel, RSM code/name), D_RSMMGR_* (4), D_MGRARE_* (4), D_MAPREN_* (6).

5.2 Silver layer — **DATA GOVERNANCE_SL** (editable source)

The **MAPPING_*** tables are the editable source that feeds the gold dimensions. They carry audit columns (**LAST_UPDATE** **TIMESTAMP_NTZ** , **UPDATED_BY** **TEXT**); the two largest also carry **LAST_SALE_DATE** (**DATE**). The **LOV_*** tables are the controlled vocabularies the procedures use to resolve codes from descriptions.

SILVER MAPPING_* TABLE	FEEDS GOLD	ROWS
MAPPING_CUSTOMER_CLASSIFICATION	D_CUSTCL_*	171,371
MAPPING_PRODUCT_CATEGORY	D_PRDCAT_*	12,954
MAPPING_OPERATOR_REPORT	D_OPEREP_*	587
MAPPING_REDISTRIBUTOR	D_MAPRED_*	5,843
MAPPING_OPERATOR_ACCOUNTS	D_MAPOPA_*	3,298
MAPPING_REDISTRIBUTOR_NAMES	D_MAPREN_*	30
MAPPING_SHIPTO_TERRITORY	D_SHPTER_*	8,042
MAPPING_TERRITORY_RSM	D_TERRSM_*	95
MAPPING_RSM_MANAGER	D_RSMMGR_*	46
MAPPING_MANAGER_AREA	D_MGRARE_*	12

There are **19 silver LOV_* tables** (e.g. **LOV_CORPORATE_ENTITY** 300, **LOV_DISTRIBUTOR** 8,318, **LOV_OPERATOR_ACCOUNT** 7,267, **LOV_TERRITORY** 207, **LOV_CHANNEL_SUBCHANNEL** 21, **LOV_BUDGET_CATEGORY** 23, plus the product-attribute LOVs).

Two are special: `LOV_REGIONAL_SALES_MANAGER` and `LOV_MANAGER` carry employee IDs (`RSM_EEID` / `MANAGER_EEID`) used to backfill the Operator Report.

Row counts differ between matched silver and gold (e.g. `MAPPING_OPERATOR_REPORT` 587 vs `D_OPEREPR` 189) because gold applies de-duplication, active/mapped filters, and reconciliation — it is **not** a row-for-row copy.

SECTION 06

Stored procedures — how to call them

ANATOMY OF A WRITE — WHAT EVERY `SP_INSERT_*` CALL DOES



This is the section most people will consult. Every editable Master Data table is written through a procedure named `SP_INSERT_<table>`. Applications, the admin UI, and ad-hoc scripts write **only** through these — never with direct DML. The procedure is the single audited write path.

6.1 General contract (applies to every procedure)

- **Pick the environment by schema.** Each procedure is deployed in **both** `PILGRIMS.DATAGOVERNANCE_SL` (production) and `PILGRIMS.DATAGOVERNANCE_SL_DEV` (development). Calling the prod copy writes prod silver+gold; calling the dev copy writes the `_DEV` mirrors. **The signature and behavior are identical.** *The examples below use the production schema; replace `DATAGOVERNANCE_SL` with `DATAGOVERNANCE_SL_DEV` to test against dev.*
- **P_DATABASE is always the first argument** (always `'PILGRIMS'`). It parameterizes the database name only — not the schema.
- **P_UPDATED_BY is always the last argument** — a free-text audit stamp (user, email, or process name) written to `UPDATED_BY`. `LAST_UPDATE` is set automatically to `CURRENT_TIMESTAMP()`.
- **All parameters are VARCHAR**. Numbers and dates are passed as strings and cast internally. Pass everything as text.
- **Blanks become 'UNMAPPED'**. Every descriptive field is normalized with `COALESCE(NULLIF(UPPER(TRIM(<param>)), ''), 'UNMAPPED')`, so NULL, empty string, and whitespace all collapse to the sentinel `UNMAPPED` (descriptions are also upper-cased and trimmed).
- **You pass descriptions, not codes.** For each description argument the procedure looks up the matching `*_CODE` in the relevant `LOV_*` table and stores both. If the description is `UNMAPPED` (or has no LOV match) the code stays `NULL`. **The value must exist in its LOV first** — otherwise the description lands but its code resolves to `NULL`.

6.2 The two behavioral patterns

1. **Incremental (the common case).** Insert the row into silver, then a single row-level `MERGE` into the matching gold table on its natural key (matched → update in place; new key → insert). Used by every mapping and LOV procedure except the Operator Report.
2. **Full-rebuild (`OPERATOR_REPORT` only).** Inside a single transaction: insert into silver, then `DELETE` the entire gold table and re- `MERGE` it from silver with dedup-by-latest and a sales-person `HAS_MAPPED` filter, followed by global backfills of `RSM_EEID`, `MANAGER_EEID`, and the corporate-entity code. The full rebuild is required because that filter logic cannot be reproduced incrementally without leaving stale rows.

6.3 Mapping procedures

SP_INSERT_MAPPING_CUSTOMER_CLASSIFICATION

The flagship product ↔ customer classification mapping. The current overload takes **20 arguments** (the 18th, `P_TERRITORY_DESCRIPTION`, was added by JBS-803 — prefer this version).

SIGNATURE

```
(P_DATABASE, P_PRODUCT, P_BRAND, P_SHIPTOPARTY, P_PAYER, P_PROFITCENTER,  
P_PRODUCTDESC, P_BRAND_DESCRIPTION, P_SHIPTOPARTY_DESCRIPTION, P_SHIP_TO_ZIP_CODE,  
P_REGION, P_PAYER_DESCRIPTION, P_CHANNEL, P_SUBCHANNEL,  
P_REDISTRIBUTOR_ACCOUNT_DESCRIPTION, P_CORPORATE_ENTITY_DESCRIPTION,  
P_DISTRIBUTOR_DESCRIPTION, P_TERRITORY_DESCRIPTION, P_LAST_SALE_DATE, P_UPDATED_BY)
```

SQL

```
CALL PILGRIMS.DATAGOVERNANCE_SL.SP_INSERT_MAPPING_CUSTOMER_CLASSIFICATION(  
  'PILGRIMS', '100123', 'PILGRIMS BRAND', '0001234567', '0007654321', 'PC1001',  
  'CHICKEN BREAST FILLET', 'PILGRIMS', 'ACME FOODS LLC', '30301',  
  'SOUTHEAST', 'ACME FOODS LLC', 'FOODSERVICE', 'K-12',  
  'US FOODS', 'SYSCO CORP', 'US FOODS', 'GEORGIA', '2026-05-31', 'test.user@example.com');
```

Resolves channel/sub-channel, redistributor, corporate-entity, distributor, and territory codes from their LOVs; writes silver `MAPPING_CUSTOMER_CLASSIFICATION`, merges gold `D_CUSTCL_CUSTOMER_CLASSIFICATION`. Merge key is `PRODUCT + SHIP_TO_PARTY + PAYER + PROFIT_CENTER` (latest brand wins; `BRAND` is intentionally **not** part of the key). A 19-argument overload without `P_TERRITORY_DESCRIPTION` still exists for older callers.

SP_INSERT_MAPPING_PRODUCT_CATEGORY

Material → product categorization (FSE category, budget category, and the six `PF_*` product-family attributes).

SIGNATURE

```
(P_DATABASE, P_MATERIAL_CODE, P_MATERIAL_DESCRIPTION, P_FSE_PRODUCT_CATEGORY,  
P_BUDGET_CATEGORY, P_PF_PROCESS_TYPE, P_PF_RAW_MATERIAL_CATEGORY, P_PF_PRODUCT_TYPE,  
P_PF_BRAND_TYPE, P_PF_BREADING_TYPE, P_PF_MEAT_TYPE, P_UPDATED_BY)
```

SQL

```
CALL PILGRIMS.DATAGOVERNANCE_SL.SP_INSERT_MAPPING_PRODUCT_CATEGORY(  
  'PILGRIMS', '100123', 'CHICKEN BREAST FILLET', 'FILLETS',  
  'FRESH', 'FURTHER PROCESSED', 'CHICKEN', 'BREAST',  
  'BRANDED', 'NON-BREADED', 'WHITE MEAT', 'test.user@example.com');
```

Resolves all eight category codes from their LOVs; writes silver `MAPPING_PRODUCT_CATEGORY`, merges gold `D_PRDCAT_PRODUCT_CATEGORY` on `MATERIAL_CODE`.

`SP_INSERT_MAPPING_OPERATOR_REPORT` (*the only full-rebuild procedure*)

The territory-matrix / operator-report mapping. The canonical overload takes **11 arguments**.

SIGNATURE

```
(P_DATABASE, P_TERRITORY_MATRIX_NUM, P_SALES_PERSON, P_OPERATOR_STATE, P_TERRITORY_NAME,  
P_CORPORATE_ENTITY_DESCRIPTION, P_CHANNEL, P_SUBCHANNEL, P_MANAGER, P_ACTIVE_FLAG, P_UPDATED_BY)
```

SQL

```
CALL PILGRIMS.DATAGOVERNANCE_SL.SP_INSERT_MAPPING_OPERATOR_REPORT(  
  'PILGRIMS', '30001', 'JANE SMITH', 'GA', 'GA-ATLANTA',  
  'US FOODS (USF)', 'FOOD SERVICE', 'CORPORATE ACCOUNTS', 'JOHN DOE', '1', 'test.user@example.com');
```

Resolves the corporate-entity code, inserts into silver `MAPPING_OPERATOR_REPORT`, then **inside a transaction** fully rebuilds gold `D_OPEREP_OPERATOR_REPORT` and backfills `RSM_EEID` (from `LOV_REGIONAL_SALES_MANAGER`), `MANAGER_EEID` (from `LOV_MANAGER`), and the corporate-entity code. `P_ACTIVE_FLAG` defaults to `1` when blank. A legacy 10-argument overload (without `P_CORPORATE_ENTITY_DESCRIPTION`) also exists; prefer the 11-argument version.

`SP_INSERT_TERRITORY_MATRIX` (*alias of the Operator Report*)

A thin alias for the operator-report write-back, kept so callers that think in "territory matrix" terms have a stable entry point. The 11-argument overload has an **identical signature** to `SP_INSERT_MAPPING_OPERATOR_REPORT` and simply delegates to it 1:1.

SIGNATURE

```
(P_DATABASE, P_TERRITORY_MATRIX_NUM, P_SALES_PERSON, P_OPERATOR_STATE, P_TERRITORY_NAME,  
P_CORPORATE_ENTITY_DESCRIPTION, P_CHANNEL, P_SUBCHANNEL, P_MANAGER, P_ACTIVE_FLAG, P_UPDATED_BY)
```

SQL

```
CALL PILGRIMS.DATAGOVERNANCE_SL.SP_INSERT_TERRITORY_MATRIX(  
  'PILGRIMS', '30001', 'JANE SMITH', 'GA', 'GA-ATLANTA',  
  'US FOODS (USF)', 'FOOD SERVICE', 'CORPORATE ACCOUNTS', 'JOHN DOE', '1', 'test.user@example.com');
```

There is no separate territory-matrix table — this writes the Operator Report. Use either name; prefer `SP_INSERT_MAPPING_OPERATOR_REPORT` in new code. Deprecated overloads (a `FLOAT`-typed 9-argument `SP_INSERT_TERRITORY_MATRIX` and `SP_INSERT_TERRITORY_MATRIX_ID`) remain for backward compatibility only.

`SP_INSERT_MAPPING_REDISTRIBUTOR`

Ship-to → redistributor mapping.

SIGNATURE

```
(P_DATABASE, P_SHIP_TO_NUMBER, P_SHIP_TO_NAME, P_DISTRIBUTOR_TYPE, P_BUYING_GROUP,  
P_CORPORATE_ENTITY_DESCRIPTION, P_DISTRIBUTOR_DESCRIPTION,  
P_CORPORATE_ACCOUNT_RESPONSIBLE, P_FIELD_SALES_RESPONSIBLE, P_UPDATED_BY)
```

```
CALL PILGRIMS.DATAGOVERNANCE_SL.SP_INSERT_MAPPING_REDISTRIBUTOR(
  'PILGRIMS', '0001234567', 'ACME FOODS LLC', 'REDISTRIBUTOR', 'UNIPRO',
  'SYSCO CORP', 'US FOODS', 'MIKE JONES', 'SARAH LEE', 'test.user@example.com');
```

Resolves buying-group, corporate-entity, and distributor codes; writes silver `MAPPING_REDISTRIBUTOR`, merges gold `D_MAPRED_MAPPING_REDISTRIBUTOR` on `SHIP_TO_NUMBER`.

SP_INSERT_MAPPING_OPERATOR_ACCOUNTS

Ship-to → corporate entity / distributor / redistributor account.

```
(P_DATABASE, P_SHIP_TO, P_SHIP_TO_DESCRIPTION, P_CORPORATE_ENTITY_DESCRIPTION,
 P_DISTRIBUTOR_DESCRIPTION, P_REDISTRIBUTOR_ACCOUNT_DESCRIPTION, P_UPDATED_BY)
```

```
CALL PILGRIMS.DATAGOVERNANCE_SL.SP_INSERT_MAPPING_OPERATOR_ACCOUNTS(
  'PILGRIMS', '1234567', 'ACME FOODS LLC', 'SYSCO CORP',
  'US FOODS', 'US FOODS', 'test.user@example.com');
```

`P_SHIP_TO` is cast to a number and left-padded to 10 digits in gold. Merges gold `D_MAPOPA_MAPPING_OPERATOR_ACCOUNTS` on the padded `SHIP_TO`.

SP_INSERT_MAPPING_REDISTRIBUTOR_NAMES

Payer → redistributor account name + corporate entity.

```
(P_DATABASE, P_PAYER, P_PAYER_DESCRIPTION, P_REDISTRIBUTOR_ACCOUNT_DESCRIPTION,
 P_CORPORATE_ENTITY_DESCRIPTION, P_UPDATED_BY)
```

```
CALL PILGRIMS.DATAGOVERNANCE_SL.SP_INSERT_MAPPING_REDISTRIBUTOR_NAMES(
  'PILGRIMS', '0007654321', 'ACME FOODS LLC', 'US FOODS', 'SYSCO CORP', 'test.user@example.com');
```

Resolves redistributor-account and corporate-entity codes; merges gold `D_MAPREN_MAPPING_REDISTRIBUTOR_NAMES` on `PAYER`.

SP_INSERT_MAPPING_SHIPTO_TERRITORY

Ship-to ZIP + sub-channel → sales territory.

```
(P_DATABASE, P_SHIP_TO_ZIP_CODE, P_SUBCHANNEL, P_TERRITORY, P_UPDATED_BY)
```

```
CALL PILGRIMS.DATAGOVERNANCE_SL.SP_INSERT_MAPPING_SHIPTO_TERRITORY(
  'PILGRIMS', '30301', 'K-12', 'GA-ATLANTA', 'test.user@example.com');
```

Resolves the territory code; merges gold `D_SHPTER_SHIPTO_TERRITORY` on `SHIP_TO_ZIP_CODE + SUBCHANNEL` .

SP_INSERT_MAPPING_TERRITORY_RSM

Territory + sub-channel → Regional Sales Manager.

(P_DATABASE, P_TERRITORY, P_SUBCHANNEL, P_RSM, P_UPDATED_BY)

SIGNATURE

```
CALL PILGRIMS.DATAGOVERNANCE_SL.SP_INSERT_MAPPING_TERRITORY_RSM(  
  'PILGRIMS', 'GA-ATLANTA', 'K-12', 'JANE SMITH', 'test.user@example.com');
```

SQL

Resolves the RSM code; merges gold `D_TERRSM_TERRITORY_RSM` on `TERRITORY + SUBCHANNEL` .

SP_INSERT_MAPPING_RSM_MANAGER

Regional Sales Manager → manager.

(P_DATABASE, P_RSM, P_MANAGER, P_UPDATED_BY)

SIGNATURE

```
CALL PILGRIMS.DATAGOVERNANCE_SL.SP_INSERT_MAPPING_RSM_MANAGER(  
  'PILGRIMS', 'JANE SMITH', 'JOHN DOE', 'test.user@example.com');
```

SQL

Merges gold `D_RSMMGR_RSM_MANAGER` on `RSM` .

SP_INSERT_MAPPING_MANAGER_AREA

Manager → area.

(P_DATABASE, P_MANAGER, P_AREA, P_UPDATED_BY)

SIGNATURE

```
CALL PILGRIMS.DATAGOVERNANCE_SL.SP_INSERT_MAPPING_MANAGER_AREA(  
  'PILGRIMS', 'JOHN DOE', 'SOUTHEAST', 'test.user@example.com');
```

SQL

Merges gold `D_MGRARE_MANAGER_AREA` on `MANAGER` .

`SP_INSERT_MAPPING_SHIPTO_EXCEPTIONS` exists in **DEV only** today (JBS-800); it is not yet promoted to production. Same calling conventions; documented here for completeness.

6.4 List-of-Value (LOV) procedures

The LOVs are the source of truth for the description → code resolution every mapping procedure relies on, so **add a value to its LOV before mapping against it**. There are 19 LOV procedures. Shared rules:

- If `P_CODE` is blank, the procedure reuses the existing code for that description or **generates the next sequential code** (e.g. `CE-0001`, `RSM001`, `T00001`).
- `P_DELETE_FLAG = '1'` removes the value from gold instead of upserting it; `'0'` upserts.
- `P_UPDATED_BY` is always last.

They come in four signature shapes:

(a) Standard — `(P_DATABASE, P_CODE, P_DESCRIPTION, P_DELETE_FLAG, P_UPDATED_BY)` — 11 procedures:

PROCEDURE	TARGET LOV TABLE
SP_INSERT_LOV_AREA	LOV_AREA
SP_INSERT_LOV_BRAND_TYPE	LOV_BRAND_TYPE
SP_INSERT_LOV_BREADING_TYPE	LOV_BREADING_TYPE
SP_INSERT_LOV_BUDGET_CATEGORY	LOV_BUDGET_CATEGORY
SP_INSERT_LOV_BUYING_GROUP	LOV_BUYING_GROUP
SP_INSERT_LOV_FSE_PRODUCT_CATEGORY	LOV_FSE_PRODUCT_CATEGORY
SP_INSERT_LOV_MEAT_TYPE	LOV_MEAT_TYPE
SP_INSERT_LOV_PROCESS_TYPE	LOV_PROCESS_TYPE
SP_INSERT_LOV_PRODUCT_TYPE	LOV_PRODUCT_TYPE
SP_INSERT_LOV_RAW_MATERIAL_CATEGORY	LOV_RAW_MATERIAL_CATEGORY
SP_INSERT_LOV_TERRITORY	LOV_TERRITORY

SQL

```
-- New territory (code auto-generated as T#####):  
CALL PILGRIMS.DATAGOVERNANCE_SL.SP_INSERT_LOV_TERRITORY(  
  'PILGRIMS', '', 'GA-ATLANTA', '0', 'test.user@example.com');
```

(b) With `P_ACCOUNT_TYPE` — `(P_DATABASE, P_CODE, P_DESCRIPTION, P_ACCOUNT_TYPE, P_DELETE_FLAG, P_UPDATED_BY)` — 4 procedures:

PROCEDURE	TARGET LOV TABLE
SP_INSERT_LOV_CORPORATE_ENTITY	LOV_CORPORATE_ENTITY
SP_INSERT_LOV_DISTRIBUTOR	LOV_DISTRIBUTOR
SP_INSERT_LOV_OPERATOR_ACCOUNT	LOV_OPERATOR_ACCOUNT
SP_INSERT_LOV_REDISTRIBUTOR_ACCOUNTS	LOV_REDISTRIBUTOR_ACCOUNTS

SQL

```
-- New corporate entity (code auto-generated as CE-NNNN):
CALL PILGRIMS.DATAGOVERNANCE_SL.SP_INSERT_LOV_CORPORATE_ENTITY(
  'PILGRIMS', '', 'SYSCO CORP', 'DISTRIBUTOR', '0', 'test.user@example.com');
```

(c) With an employee ID — 2 procedures (the EEID feeds the Operator Report backfill). Same as the standard shape, with an employee-ID parameter inserted after `P_DESCRIPTION`:

- `SP_INSERT_LOV_REGIONAL_SALES_MANAGER` → `LOV_REGIONAL_SALES_MANAGER` — (`P_DATABASE`, `P_CODE`, `P_DESCRIPTION`, `P_RSM_EEID`, `P_DELETE_FLAG`, `P_UPDATED_BY`)
- `SP_INSERT_LOV_MANAGER` → `LOV_MANAGER` — (`P_DATABASE`, `P_CODE`, `P_DESCRIPTION`, `P_MANAGER_EEID`, `P_DELETE_FLAG`, `P_UPDATED_BY`)

SQL

```
-- Regional sales manager with EEID (code auto-generated as RSMNNN):
CALL PILGRIMS.DATAGOVERNANCE_SL.SP_INSERT_LOV_REGIONAL_SALES_MANAGER(
  'PILGRIMS', '', 'JANE SMITH', '00123456', '0', 'test.user@example.com');
```

(d) Composite (7 arguments) — 2 procedures:

- `SP_INSERT_LOV_CHANNEL_SUBCHANNEL` → `LOV_CHANNEL_SUBCHANNEL` — (`P_DATABASE`, `P_CHANNEL_CODE`, `P_CHANNEL_DESCRIPTION`, `P_SUBCHANNEL_CODE`, `P_SUBCHANNEL_DESCRIPTION`, `P_DELETE_FLAG`, `P_UPDATED_BY`)
- `SP_INSERT_LOV_CUSTOMER_CORP` → `LOV_CUSTOMER_CORP` — (`P_DATABASE`, `P_CORPORATE_ENTITY_CODE`, `P_CORPORATE_ENTITY_DESCRIPTION`, `P_DISTRIBUTOR_CODE`, `P_DISTRIBUTOR_DESCRIPTION`, `P_DELETE_FLAG`, `P_UPDATED_BY`)

SQL

```
-- Channel + sub-channel pair:
CALL PILGRIMS.DATAGOVERNANCE_SL.SP_INSERT_LOV_CHANNEL_SUBCHANNEL(
  'PILGRIMS', 'F', 'FOODSERVICE', 'K12', 'K-12', '0', 'test.user@example.com');
```

SECTION 07

Conventions & validation

- **UNMAPPED sentinel.** Any blank description is stored as `UNMAPPED` (code `NULL`). Downstream filters and the Operator Report `HAS_MAPPED` logic depend on this — don't substitute your own placeholder.
- **Code ↔ description.** Callers supply descriptions; codes are LOV-resolved and stored alongside. Re-mapping a description in the LOV is reconciled on the next procedure call / ETL run.
- **Comparing two tables** (silver vs gold, dev vs prod, vs legacy): row count + order-insensitive content hash.


```
SELECT 'A' AS SRC, COUNT(*) AS CNT, HASH_AGG(*) AS H FROM <table_a>
UNION ALL
SELECT 'B', COUNT(*), HASH_AGG(*) FROM <table_b>;
```

`HASH_AGG(*)` compares values, not column names — use `INFORMATION_SCHEMA.COLUMNS` for structural diffs and `EXCEPT` (both directions) for row-level diffs.

SECTION 08

Operational notes & gotchas

- **MAPPING_CUSTOMER_CLASSIFICATION has multiple writers.** The daily ETL (`UPDATED_BY = 'ETL'`), bulk mapping-file loads, and the app (via the SP) all write it. The ETL's silver insert applies the JBS-775 filter (`YEAR(POSTINGDATE) >= 2024 + 6` added profit centers); after any manual backfill, the `LAST_SALE_DATE` and `REGION` enrichment UPDATES must be re-run, or Qlik shows blank Last Sales Date / State.
- **CONSOLIDATED_ACCOUNT_LIST has no SP_INSERT_***. It is rebuilt only by the daily batch (DELETE + full INSERT), as the union of the GD `LOV*` tables and the operator/distributor/redistributor mappings, with addresses resolved from SAP `zev_customer` (D_CUSTCL ship-to first, MAPOPA ship-to as fallback).
- **Changing the ETL.** ETL behavior is changed by editing the source SQL (`Master Data/PROD ETL.txt`) and **regenerating** the `SP_MASTER_DATA_ETL` procedure with the deterministic builder (`Master Data/orchestration/build/generate_etl.py`), then redeploying. Interactive single-row write behavior is changed in the `SP_INSERT_*` DDL.
- **Two prod writers to gold.** The daily ETL and the prod `SP_INSERT_*` family can both write prod gold; the gold rebuilds are idempotent and reconcile from silver, so they coexist safely.
- **Known data gap:** `D_MAPRED.MAPRED_BUYING_GROUP` is effectively `UNMAPPED` and `LOV_BUYING_GROUP` carries only `UNIPRO` , so the redistributor buying-group code is largely unpopulated — a data gap, not a code bug.

Glossary

TERM	MEANING
Silver (DATAGOVERNANCE_SL)	Editable source-of-truth tables (MAPPING_*, LOV_*).
Gold (DATAGOVERNANCE_GD)	Curated, consumption-ready dimensions (D_*, LOV*, CONSOLIDATED_ACCOUNT_LIST).
LOV	List of Values — a controlled vocabulary (valid codes + descriptions).
LEGACY_PPC.PPC_MASTER_DATA	The dbt-published, read-only copy of gold that consumers query.
EEID	Employee ID, backfilled into the Operator Report for RSM and manager.
HAS_MAPPED	Operator-Report filter that keeps a sales person's rows out of gold until at least one is fully mapped.
UNMAPPED	Sentinel stored when a description is blank (code stays NULL).
MST	Mountain Standard Time — the daily ETL runs at 06:00 MST.